Observer Design Pattern

1. What is Observer Design Pattern?

The **Observer Design Pattern** defines a **one-to-many dependency** between objects such that when one object (the *Subject*) changes its state, all its dependent objects (the *Observers*) are **automatically notified and updated**.

This pattern is widely used in event-driven systems, GUIs, and real-time data applications.

2. Why Use the Observer Pattern?

The main goal of the Observer pattern is to achieve **loose coupling** between the subject and its observers.

It helps in:

- Decoupling objects
- Automatic event notification
- Dynamic addition and removal of observers

The subject does not need to know:

- Who the observers are
- How many observers exist

3. Core Components of Observer Pattern

- **Subject**: Maintains state and notifies observers
- **Observer**: Defines an update interface
- **ConcreteSubject**: Implements Subject and stores state
- **ConcreteObserver**: Implements Observer and reacts to updates

4. Real-World Example

Weather Monitoring System

- Weather Station → Subject
- Displays → Observers
- Any change in weather data automatically updates all displays

This avoids continuous polling and improves efficiency.

5. Observer Pattern Implementation in Java

5.1 Step 1: Subject Interface

```
interface Subject {
    void addObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}
```

5.2 Step 2: Observer Interface

```
interface Observer {
    void update(float temperature, float humidity);
}
```

5.3 Step 3: Concrete Subject (WeatherStation)

```
import java.util.ArrayList;
import java.util.List;

class WeatherStation implements Subject {

    private List<Observer> observers = new ArrayList<>();
    private float temperature;
    private float humidity;

    public void addObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o : observers) {
            o.update(temperature, humidity);
        }
```

```
    }

    public void setMeasurements(float temp, float hum) {
        this.temperature = temp;
        this.humidity = hum;
        notifyObservers();
    }
}
```

5.4 Step 4: Concrete Observer (Display)

```
class WeatherDisplay implements Observer {

    public void update(float temperature, float humidity) {
        System.out.println(
            "Current␣conditions:␣" + temperature +
            "␣C␣and␣" + humidity + "%␣humidity"
        );
    }
}
```

5.5 Step 5: Testing the Pattern

```
public class ObserverDemo {

    public static void main(String[] args) {

        WeatherStation station = new WeatherStation();

        WeatherDisplay d1 = new WeatherDisplay();
        WeatherDisplay d2 = new WeatherDisplay();

        station.addObserver(d1);
        station.addObserver(d2);

        station.setMeasurements(25.5f, 65.0f);
        station.setMeasurements(27.3f, 70.0f);
    }
}
```

6. Advantages of Observer Pattern

- Loose coupling between subject and observers
- Supports Open/Closed Principle
- Dynamic subscription management

- Ideal for event-driven architectures

7. Disadvantages of Observer Pattern

- Memory leaks if observers are not removed properly
- Performance overhead with many observers
- Order of notification is not guaranteed

> Improper observer management can lead to serious memory and performance issues.

8. When to Use Observer Pattern

Use Observer Pattern when:

- An object needs to notify others about state changes
- Implementing event listeners
- Handling real-time data updates

Common applications:

- GUI event handling
- Stock market tickers
- Messaging systems (Kafka, RabbitMQ)

9. Exam Tip

> If asked: *"What problem does Observer Pattern solve?"* Answer: It solves the problem of **maintaining consistency between related objects without tight coupling**.

10. Conclusion

The Observer Design Pattern is a fundamental behavioral pattern for building scalable, event-driven systems. It promotes loose coupling, flexibility, and dynamic behavior, making it essential in modern software design.